

UNIT II

INTRODUCTION TO GENERATIVE AI AND LLMs

Syllabus : Sequence Modelling – Transformer Architecture – Self-Attention and Multi-head Attention Mechanisms – Positional Encoding – Encoder and Decoder Models – BERT, GPT, T5, LLaMA Architectures – Tokenization Techniques – Embeddings and Vector Representations – Scaling Laws in Large Language Models – Mixture of Experts (MoE) Architectures – Emergent Capabilities in Large Language Models.

SEQUENCE MODELLING

Sequence Modeling is a machine learning approach used to analyze and predict data where the order of observations matters. Instead of treating inputs as independent, sequence models learn patterns across time or position.

Sequence modeling is a branch of machine learning and deep learning that focuses on processing and analyzing data that occur in a specific order. Unlike conventional machine learning models that assume all input samples are independent of each other, sequence models exploit the dependencies among elements in a sequence to learn meaningful patterns and make accurate predictions.

In many real-world applications, the order of data points carries significant information. For example, in a sentence, the meaning of a word often depends on the words that precede or follow it. Similarly, in stock market prediction, previous stock prices influence future trends. Sequence modeling techniques are designed to capture these temporal or positional relationships effectively.

Need for Sequence Modeling

Traditional machine learning algorithms such as Decision Trees, Support Vector Machines, and Linear Regression generally assume that input observations are independent and identically distributed. However, this assumption is not valid for sequential data.

Consider the sentence:

"The cat sat on the ____."

The missing word is most likely mat because the previous words provide context. A conventional classifier cannot effectively utilize this contextual information, whereas a sequence model can.

Sequence modeling becomes essential because it enables systems to:

- Understand contextual relationships among data points.
- Capture temporal dependencies.
- Predict future events based on historical information.
- Generate meaningful sequences such as text, speech, or music.
- Process variable-length input and output sequences.

Markov Model

Markov models are powerful tools used in various fields such as finance, biology, and natural language processing. They are particularly useful for modeling systems where the next state depends only on the current state and not on the sequence of events that preceded it. A Markov Model is a way of describing how something changes over time. It works by saying that the chance of something happening next only depends on what's happening right now, not on how it got to this point.

A **Markov Model** is a stochastic model that describes a sequence of possible events where the probability of each event depends only on the state attained in the previous event. In simple terms, a Markov Model predicts the next state of a system based solely on the current state.

The defining property of a Markov process is known as the **Markov Property**.

Markov Property

The Markov Property states that:

The future state of a process depends only on its present state and is independent of all past states.

Mathematically,

$$P(X_{t+1}|X_t, X_{t-1}, X_{t-2}, \dots, X_0) = P(X_{t+1}|X_t)$$

where:

- X_t = current state
- X_{t+1} = future state
- $X_0, X_1, \dots, X_{t-1}$ = previous states

This assumption significantly simplifies the modeling of sequential data.

Types of Markov models

Two commonly applied types of Markov model are used when the system being represented is autonomous -- that is, when the system isn't influenced by an external agent. These are as follows:

1. Markov Chains

Markov models are often visualized as Markov chains, where each state is represented as a node, and the transitions between states are depicted as directed edges. The probability of transitioning from one state to another is associated with the corresponding edge.

Imagine we have three types of weather: sunny, cloudy, and rainy. We want to predict the weather for the next day based on the current day. We can use a Markov Chain to represent this.

Here's how we visualize it:

- **States:** Sunny, Cloudy, Rainy (represented as nodes)
- **Transitions:** Arrows between the states represent the likelihood of moving from one weather type to another.

Now, let's consider the probabilities associated with each transition:

- If it's sunny today, there's a 70% chance it will be sunny again tomorrow (arrow pointing to sunny).
- If it's sunny today, there's a 20% chance it will be cloudy tomorrow (arrow pointing to cloudy).
- If it's sunny today, there's a 10% chance it will be rainy tomorrow (arrow pointing to rainy).
- This pattern continues for each weather type, indicating the probabilities of transitioning to other states.

So, a Markov Chain for this weather model visually shows how likely it is to move from one weather state to another. It simplifies the prediction process by focusing only on the current state and the associated probabilities for transitioning to the next state.

2. Hidden Markov Models (HMMs)

Suitable for systems with unobservable states, HMMs include observations and observation likelihoods for each state. They find application in thermodynamics, finance, and pattern recognition.

- In speech recognition, words spoken form the observable states, but the actual phonemes (basic sound units) producing those words are hidden. Each word has a sequence of hidden phonemes that generate it.
- **Observable states:** Words spoken ("hello," "goodbye," etc.)
- **Hidden states:** Phonemes making up each word ("h-e-l-l-o" for "hello")

The aim is to determine the sequence of hidden states (phonemes) based on the observable states (spoken words). By analyzing large datasets of spoken words and their corresponding phoneme sequences, HMMs help identify the most probable hidden states (phonemes) that produce observed sequences (words).

Another two commonly applied types of Markov model are used when the system being represented is controlled that is, when the system is influenced by a decision-making agent.

3. Markov Decision Processes (MDPs)

Model decision-making in discrete, stochastic, sequential environments where an agent relies on reliable information. Applications span artificial intelligence, economics, and behavioral sciences.

Example: Autonomous Drone Delivery

Consider an autonomous drone delivering packages in a city. Markov Decision Processes (MDPs) model the decision-making, where:

- **States:** Represent various city locations (e.g., intersections, delivery points).
- **Actions:** Include drone movements and decisions (e.g., fly left, fly right, deliver package).
- **Result:** Reflect positive/negative feedback based on successful/unsuccessful deliveries, considering factors like delivery time.

MDPs guide the drone's decisions for an optimized delivery route, factoring in variables like traffic, package priority, and deadlines. Sequential decision-making involves transitions between states influenced by chosen actions, ensuring efficient and timely deliveries.

4. Partially Observable Markov Decision Processes (POMDPs)

Applied when the decision-making agent lacks reliable information intermittently. Robotics and machine maintenance are common domains for POMDPs.

Example: Autonomous Vacuum Cleaning Robot

Imagine an autonomous vacuum cleaning robot operating in a household environment. The decision-making process can be effectively modeled using Partially Observable Markov Decision Processes (POMDPs). In this scenario:

- **States:** Represent possible locations in the house (e.g., living room, bedroom).
- **Actions:** Encompass robot movements and cleaning decisions (e.g., move forward, turn, start cleaning).
- **Observations:** Include sensor readings, capturing information about the environment (e.g., dirt level, obstacles).
- **Result:** Reflect positive or negative feedback based on the effectiveness of cleaning and navigation.

POMDPs enable the robot to make decisions considering incomplete information due to sensor limitations. The robot's actions and transitions between states are influenced by sensor observations, allowing for adaptive and efficient cleaning in dynamically changing environments.

Transformers

Transformer is a neural network architecture used for various machine learning tasks, especially in natural language processing and computer vision. It focuses on understanding relationships within data to process information more effectively.

- Uses attention mechanisms to capture relationships between inputs
- Processes entire sequences at once instead of step by step
- Improves performance on tasks involving context and dependencies
- Widely used across NLP, vision and other AI applications

Need For Transformers Model

Transformer architecture uses attention to process an entire sentence at once instead of reading words sequentially. This helps overcome limitations of models like RNNs and LSTMs that process data step by step.

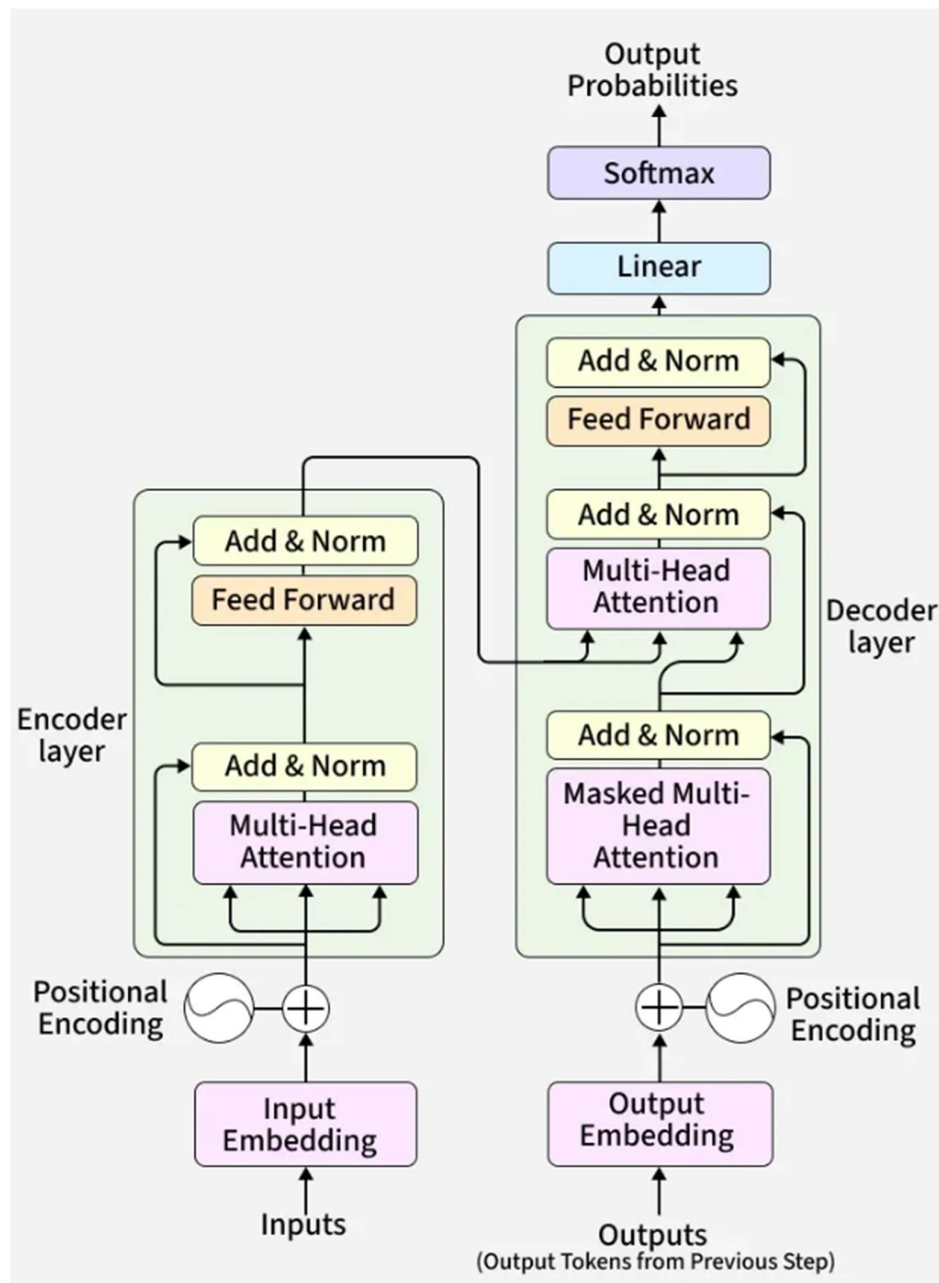
Traditional models like RNNs (Recurrent Neural Networks) suffer from the vanishing gradient problem which leads to long-term memory loss.

RNNs process text sequentially meaning they analyze words one at a time.

While adding more memory cells in LSTMs (Long Short-Term Memory networks) helped address the vanishing gradient issue they still process words one by one. This sequential processing means LSTMs can't analyze an entire sentence at once.

- Traditional Seq2Seq models compress the entire input into a single fixed-size context vector.
- This creates an **information bottleneck**, especially for long sequences.
- Important context may be lost during compression, reducing performance.
- Transformers solve this by using attention to access all tokens directly without compression.





1. Self Attention Mechanism

The self attention mechanism allows transformers to determine which words in a sentence are most relevant to each other. This is done using a scaled dot-product attention approach. Each word in a sequence is mapped to three vectors:

- Query (Q)
- Key (K)
- Value (V)

Attention scores are computed as: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

These scores determine how much attention each word should pay to others.

2. Multi-Head Attention

Instead of one attention mechanism, transformers use multiple attention heads running in parallel. Each head captures different relationships or patterns in the data, enriching the model's understanding.

3. Positional Encoding

Unlike RNNs, transformers lack an inherent understanding of word order since they process data in parallel. To solve this problem Positional Encodings are added to token embeddings providing information about the position of each token within a sequence.

4. Position-wise Feed-Forward Networks

The Feed-Forward Networks consist of two linear transformations with a ReLU activation. It is applied independently to each position in the sequence. This transformation helps refine the encoded representation at each position.

Mathematically:

$$FFN(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

5. Add & Norm (Residual Connections and Layer Normalization)

Each sub-layer in the transformer (such as self-attention and feed-forward networks) is followed by an Add & Norm step.

- **Residual Connection (Add):** The input of a layer is added to its output. This helps in preserving information and prevents vanishing gradients in deep networks.
- **Layer Normalization (Norm):** The combined output is normalized to stabilize training and improve convergence.

This structure ensures better gradient flow and allows training of deep transformer models efficiently.

6. Embeddings

Transformers cannot work with raw words as they need numbers. So, each input token (word or subword) is converted into a vector, called an embedding.

- Both encoder input tokens and decoder input tokens are converted into embeddings.
- These embeddings are trainable, meaning the model learns the best numeric representation for each token.
- The same weight matrix is shared for Encoder embeddings, Decoder embeddings and the final linear layer before softmax
- The embeddings are scaled by model to keep values stable before adding positional encoding.

7. Encoder-Decoder Architecture

The encoder-decoder structure is key to transformer models. The encoder processes the input sequence into a vector, while the decoder converts this vector back into a sequence. Each encoder and decoder layer includes self-attention and feed-forward layers.

For example, a French sentence "Je suis étudiant" is translated into "I am a student" in English.

Transformers apply attention in three different places:

1. Encoder Self-Attention

- Q, K, V all come from the encoder's previous layer.
- Every word can attend to every other word in the input.
- This helps the encoder understand full context (long-range meaning).

2. Decoder Self-Attention (Masked)

- Q, K, V all come from the decoder's previous layer.
- Future tokens are masked (blocked), so each position only sees previous tokens.
- This keeps decoding auto-regressive i.e the model predicts one word at a time.

3. Encoder-Decoder Attention

- Queries come from the decoder.
- Keys and Values come from the encoder output.
- This lets the decoder look at important parts of the input sentence while generating output.

Together, these three attention types allow the transformer to read the entire input at once and then generate outputs step-by-step with full context.

8. Softmax Layer for Output Prediction

After the decoder processes the sequence, it must predict the next token.

- The decoder output is passed through a linear layer (whose weights are shared with embeddings).
- Then the softmax function converts these scores into probabilities.
- The token with the highest probability becomes the predicted next word.

Applications

- **NLP Tasks:** Transformers are used for machine translation, text summarization, named entity recognition and sentiment analysis.
- **Speech Recognition:** They process audio signals to convert speech into transcribed text.
- **Computer Vision:** Transformers are applied to image classification, object detection and image generation.
- **Recommendation Systems:** They provide personalized recommendations based on user preferences.
- **Text and Music Generation:** Transformers are used for generating text like articles and composing music.

Limitations

- Require high computational power, making them expensive to train
- Consume large memory, especially for long sequences
- Need large datasets to perform well
- Have input length limits, making very long text handling difficult
- Can behave like a black box, reducing interpretability
- Risk of overfitting if not properly regularized

SELF-ATTENTION MECHANISM

The Self-Attention mechanism is the fundamental component of the Transformer architecture and is one of the key innovations behind models such as BERT, GPT, T5, and LLaMA. It enables a model to understand the importance of every word in a sentence by examining its relationship with all other words in the same sequence. Unlike traditional Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks, which process words sequentially, self-attention processes all words simultaneously while dynamically determining which words should receive greater attention during representation learning.

The primary objective of self-attention is to generate a context-aware representation of every word. Rather than assigning a fixed embedding to a word, self-attention allows the representation of a word to change depending on the surrounding context. Consequently, the meaning of a word becomes dependent on the complete sentence rather than the word itself. For example, consider the following two sentences:

- The bank approved the loan.
- The children played near the bank of the river.

Although the word "bank" appears in both sentences, its meaning is entirely different. A traditional embedding technique such as Word2Vec or GloVe would assign the same vector to the word "bank." However, self-attention enables BERT to generate two different contextual representations by analyzing the surrounding words such as loan, approved, river, and played. This contextual understanding is one of the major strengths of Transformer-based language models.

Natural language contains long-range dependencies where the interpretation of one word depends on another word that may be located far away in the sentence. Traditional sequential models often struggle to capture these dependencies efficiently.

Consider the sentence:

The animal did not cross the street because it was tired.

To correctly understand the meaning of the pronoun "it," the model must determine whether it refers to animal or street. Humans naturally associate "it" with "animal." Self-attention allows the model to establish this relationship by directly comparing the word it with every other word in the sentence.

Similarly, consider:

The trophy does not fit into the suitcase because it is too large.

Understanding whether it refers to the trophy or the suitcase requires semantic reasoning. Self-attention enables the model to identify these dependencies regardless of the distance between the words.

Working of the Self-Attention Mechanism

The operation of self-attention begins after the input sentence has been tokenized and converted into embeddings. Each token initially consists of the sum of:

- Token Embedding
- Position Embedding
- Segment Embedding (in BERT)

Suppose the input sentence is:

The student is learning Artificial Intelligence.

After embedding, each word is represented by a numerical vector.

Rather than immediately processing these vectors, self-attention first transforms every input embedding into **three different representations** known as:

- Query (Q)
- Key (K)
- Value (V)

These vectors are obtained through learned linear transformations.

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

where:

- X is the input embedding matrix.
- W_Q is the learned weight matrix for queries.
- W_K is the learned weight matrix for keys.
- W_V is the learned weight matrix for values.

These weight matrices are learned automatically during training using backpropagation.

Query, Key, and Value

The concepts of Query, Key, and Value are inspired by information retrieval systems.

Imagine a digital library.

- A person asks a question.
- Every book has a title describing its contents.
- After identifying the relevant book, the reader extracts the actual information.

Similarly,

Query

The Query represents what the current word is searching for.
It asks,
"Which other words are important for understanding me?"

Key

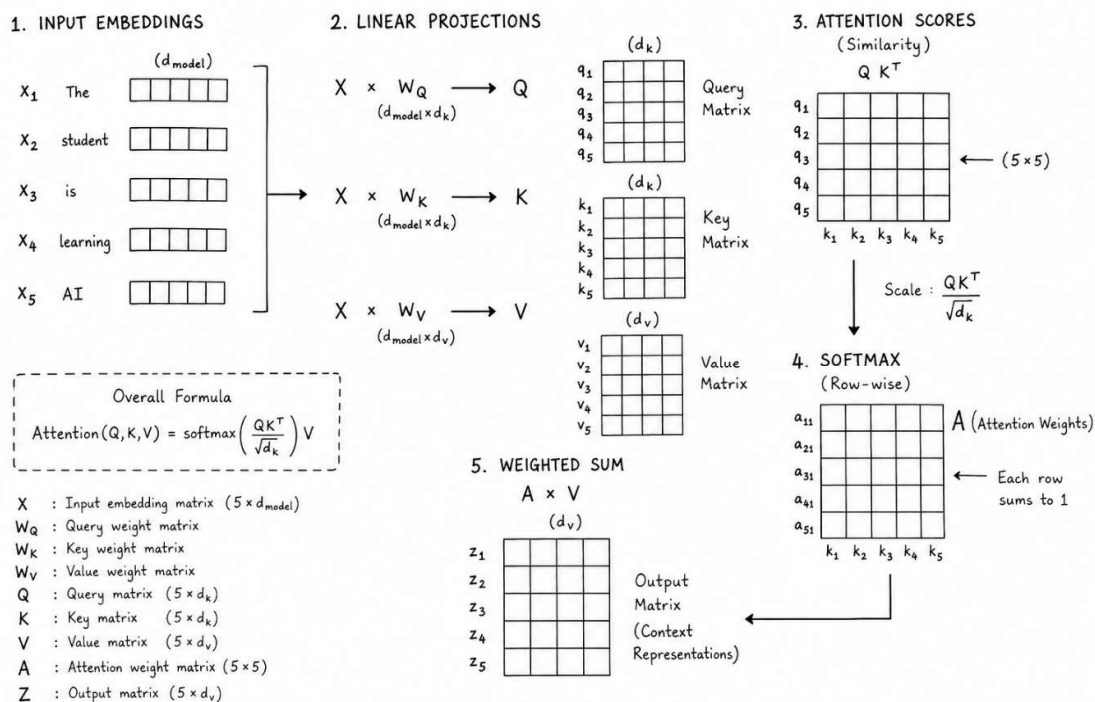
The Key represents what information each word contains.
Every word provides a key describing the type of information it carries.

Value

The Value represents the actual information that will be transferred after determining the importance of each word.

Thus,

- Query performs the search.
- Key determines relevance.
- Value provides the information.



Step 1: Computing Attention Scores

Suppose we want to update the representation of the word learning.
Its Query vector is compared with the Key vectors of every word in the sentence.
For example,
learning
is compared against
The
student
is
learning
Artificial
Intelligence

The similarity is measured using the dot product.

Mathematically,

$$\text{Score} = QK^T$$

A larger score indicates that the two words are more closely related.

For example,

Compared Word Attention Score

The	0.12
student	0.44
is	0.37
learning	0.61
Artificial	0.90
Intelligence	0.95

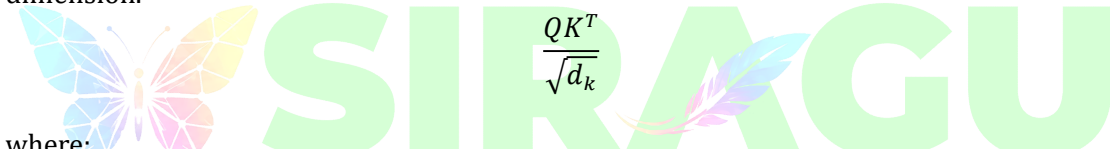
Notice that the highest scores are assigned to Artificial and Intelligence, because these words are highly relevant for understanding learning in this sentence.

Step 2: Scaling the Scores

The dot-product values can become very large when the embedding dimension increases. Large values cause the Softmax function to produce extremely sharp probability distributions, making the model difficult to train.

To prevent this problem, the attention scores are divided by the square root of the key dimension.

where:



$$\frac{QK^T}{\sqrt{d_k}}$$

- d_k represents the dimension of the key vector.

This scaling stabilizes the gradients and improves training efficiency.

Step 3: Applying the Softmax Function

The scaled attention scores are converted into probabilities using the Softmax function.

$$\text{Attention Weights} = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right)$$

Suppose the scaled scores are

Word	Score
The	0.10
student	0.35
is	0.25
learning	0.40
Artificial	1.80
Intelligence	2.00

After applying Softmax,

Word	Attention Weight
The	0.03
student	0.07

Word	Attention Weight
is	0.05
learning	0.08
Artificial	0.36
Intelligence	0.41

The weights now sum to 1.

These values indicate the importance of each word while updating the representation of learning.

Step 4: Computing the Weighted Sum

The attention weights are multiplied by the corresponding Value vectors.

$$\text{Output} = \text{Attention Weights} \times V$$

The resulting vector becomes the new representation of the word.

Instead of representing only the word learning, the new embedding now contains information from the entire sentence.

Thus, the output becomes context-aware rather than context-independent.

Complete Self-Attention Formula

The complete self-attention computation is represented as:

This formula summarizes the entire self-attention mechanism:

- Compute Query–Key similarities.
- Scale the scores.
- Apply Softmax to obtain attention weights.
- Multiply the weights with the Value vectors.
- Generate a new contextual representation.

Example of Self-Attention

Consider the sentence:

The cat sat on the mat because it was tired.

Suppose the model is processing the word it.

Instead of considering only nearby words, self-attention compares it with every word:

Word	Attention Weight
The	Low
cat	Very High
sat	Medium
on	Low
mat	Low
because	Medium
was	Medium
tired	High

Since cat receives the highest attention weight, the updated representation of it strongly reflects the meaning of cat.

Thus, the model correctly understands that it refers to the cat.

Advantages of Self-Attention

The self-attention mechanism offers several important advantages over traditional sequential architectures. It captures long-range dependencies by allowing every word to directly interact

with every other word in the sentence, regardless of their distance. Since all tokens are processed simultaneously, computation can be parallelized efficiently on modern GPUs, resulting in significantly faster training than recurrent models. Self-attention also generates contextual embeddings, meaning that the representation of a word changes depending on its surrounding words, allowing the model to distinguish between different meanings of the same word. Furthermore, by assigning different attention weights to different tokens, the model automatically focuses on the most informative parts of the sentence, leading to improved performance across tasks such as machine translation, question answering, text classification, summarization, and conversational AI.

MULTI-HEAD ATTENTION MECHANISM

Multi-Head Attention is an extension of the self-attention mechanism used in Transformer architectures. Instead of computing a single attention operation over the input sequence, the model performs several attention operations in parallel. Each parallel attention operation is called an attention head.

The purpose of using multiple heads is to allow the model to learn different relationships among tokens at the same time. A single attention head produces only one attention distribution for each token. This may not be sufficient to capture all the syntactic, semantic, positional, and contextual dependencies present in a sequence. Multi-head attention overcomes this limitation by projecting the same input into multiple lower-dimensional representation spaces and computing attention independently in each space.

The outputs of all heads are then combined to form a richer contextual representation.

Input Representation

Assume that the input sequence contains n tokens. After token embedding and positional encoding, the input is represented by the matrix:

$$X \in \mathbb{R}^{n \times d_{\text{model}}}$$

where:

- n is the number of tokens in the sequence.
- d_{model} is the dimensionality of each token representation.
- Each row of X represents one token.

Formation of Query, Key and Value for Multiple Heads

In ordinary self-attention, Query, Key and Value are generated using three learned matrices:

$$Q = XW^Q$$

$$K = XW^K$$

$$V = XW^V$$

In multi-head attention, each head has its own Query, Key and Value projection matrices.

For the i^{th} head:

$$Q_i = XW_i^Q$$

$$K_i = XW_i^K$$

$$V_i = XW_i^V$$

where:

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

$$W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

$$W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

The subscript i indicates that each attention head uses a different set of trainable weight matrices.

Therefore, even though all heads receive the same input matrix X , they generate different Query, Key and Value matrices.

Dimensionality of Each Head

Suppose the model contains h attention heads. The model dimension is usually divided equally among the heads:

$$d_k = d_v = \frac{d_{\text{model}}}{h}$$

Query-Key Matrix Multiplication

The Query matrix is multiplied by the transpose of the Key matrix:

$$Q_i K_i^T$$

The dimensions are:

$$Q_i \in \mathbb{R}^{n \times d_k}$$

$$K_i^T \in \mathbb{R}^{d_k \times n}$$

Therefore:

$$Q_i K_i^T \in \mathbb{R}^{n \times n}$$

The resulting matrix is called the attention score matrix.

Each element in the matrix represents the similarity between one token's Query vector and another token's Key vector.

For example, the element in row r and column c is:

$$s_{rc} = q_r \cdot k_c$$

This score measures how strongly token r should attend to token c .

Each row represents one querying token, and each column represents one key token.

Scaling the Attention Scores

The attention scores are divided by:

$$\sqrt{d_k}$$

Therefore, the scaled score matrix becomes:

$$\frac{Q_i K_i^T}{\sqrt{d_k}}$$

The scaling operation is necessary because the dot product increases in magnitude as the vector dimension increases. Without scaling, large scores may cause the Softmax function to produce extremely concentrated probability distributions. This can lead to very small gradients and unstable training.

Dividing by $\sqrt{d_k}$ keeps the score values within a manageable numerical range.

POSITIONAL ENCODING

Positional Encoding is the mechanism used in Transformer models to represent the order and position of tokens in a sequence. It is necessary because the Transformer architecture does not process words one after another in a recurrent manner. Instead, all tokens are processed

in parallel. Therefore, without additional position information, the model would know which tokens are present but would not naturally know their sequence order.

For example, the sentences contain almost the same words, but their meanings are different because the word order is different. A Transformer that receives only token embeddings may treat the input as a set of words rather than an ordered sequence. Positional encoding solves this problem by adding a unique position-dependent vector to each token embedding.

Recurrent neural networks naturally preserve sequence order because they process tokens one by one. The hidden state at a particular time step depends on the previous hidden state. Therefore, token order is embedded in the computation itself.

Transformers do not use recurrence. The self-attention mechanism compares every token with every other token simultaneously. This parallel computation improves training efficiency, but it removes the natural notion of sequence order.

Consider an input sequence represented by the embedding matrix:

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}$$

Each row corresponds to one token embedding. If these embeddings are passed directly to self-attention, the attention mechanism can compare token meanings, but it cannot independently determine whether a token occurred first, second, or tenth.

Positional encoding introduces this missing information.

Types of Positional Encoding

There are two major approaches:

1. Fixed positional encoding
2. Learned positional embedding

1. Fixed Sinusoidal Positional Encoding

The original Transformer model used sinusoidal positional encoding. These encodings are generated using sine and cosine functions.

For position pos and embedding dimension index i :

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

The even dimensions use sine functions, while the odd dimensions use cosine functions.

Here:

- pos is the token position.
- i is the dimension index.
- d_{model} is the embedding size.

For example, if the model dimension is 512, every position receives a vector of length 512.

Different dimensions use different frequencies. Some dimensions vary slowly across positions, while others vary rapidly.

This creates a unique positional pattern for every token position.

The low-frequency dimensions represent broad positional changes, while high-frequency dimensions help distinguish nearby positions.

Because sine and cosine functions are continuous and structured, the model can also learn relative positional relationships.

For instance, the encoding of position $pos + k$ can be expressed as a linear transformation of the encoding at position pos . This property helps the model infer relative distances between tokens.

Example of Sinusoidal Encoding

Consider a very small positional vector with four dimensions.

For position 0:

$$PE(0) = [0, 1, 0, 1]$$

because:

$$\sin(0) = 0$$

and

$$\cos(0) = 1$$

For position 1, the values change according to different frequencies:

$$PE(1) = [\sin(1), \cos(1), \sin(1/100), \cos(1/100)]$$

For position 2:

$$PE(2) = [\sin(2), \cos(2), \sin(2/100), \cos(2/100)]$$

Each position therefore obtains a distinct vector.

Positional Encoding Matrix

For a sequence of length n , all positional vectors form a matrix:

$$P \in \mathbb{R}^{n \times d_{\text{model}}}$$

The token embedding matrix is:

$$E \in \mathbb{R}^{n \times d_{\text{model}}}$$

The final input matrix is:

$$X = E + P$$

Both matrices must have the same shape so that they can be added element by element.

For a sequence of 10 tokens and model dimension 512:

$$E: 10 \times 512$$

$$P: 10 \times 512$$

$$X: 10 \times 512$$

The sequence length and embedding dimension remain unchanged.

2. Learned Positional Embeddings

Some Transformer models do not use fixed sinusoidal encodings. Instead, they use learned positional embeddings.

In this approach, every possible position has a trainable vector.

For example:

$$P_0, P_1, P_2, \dots, P_{511}$$

During model training, these vectors are updated using backpropagation in the same way as token embeddings.

BERT uses learned positional embeddings.

If the maximum sequence length is 512, BERT maintains an embedding table with 512 positional vectors. Each vector has the same dimension as the hidden representation.

For BERT Base:

$$P \in \mathbb{R}^{512 \times 768}$$

The positional vector corresponding to each token position is selected and added to the token embedding.

BERT (BIDIRECTIONAL ENCODER REPRESENTATIONS FROM TRANSFORMERS)

BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained deep learning language model developed by researchers at Google AI in 2018. It is based on the Transformer Encoder architecture and was designed to overcome the limitations of earlier Natural Language Processing (NLP) models by learning the contextual meaning of words from both the left and right sides simultaneously.

Unlike traditional language models, which process text sequentially in one direction (left-to-right or right-to-left), BERT processes the entire sentence simultaneously. This enables it to understand the meaning of a word based on its complete surrounding context.

BERT is not designed to generate text. Instead, it is primarily a language understanding model, making it highly effective for tasks such as text classification, sentiment analysis, question answering, named entity recognition, semantic similarity, and document retrieval. The introduction of BERT represented a major advancement in NLP because it demonstrated that a single pre-trained model could achieve state-of-the-art performance across many language understanding tasks with minimal task-specific modifications.

The term Bidirectional means that BERT considers information from both directions of a sentence during training.

Traditional models process text as:

Left → Right

or

Right → Left

BERT processes both directions simultaneously.

BERT analyses

$$Representation(executes) = f(Left\ Context, Right\ Context)$$

instead of

$$Representation(executes) = f(Left\ Context)$$

This enables much richer contextual understanding.

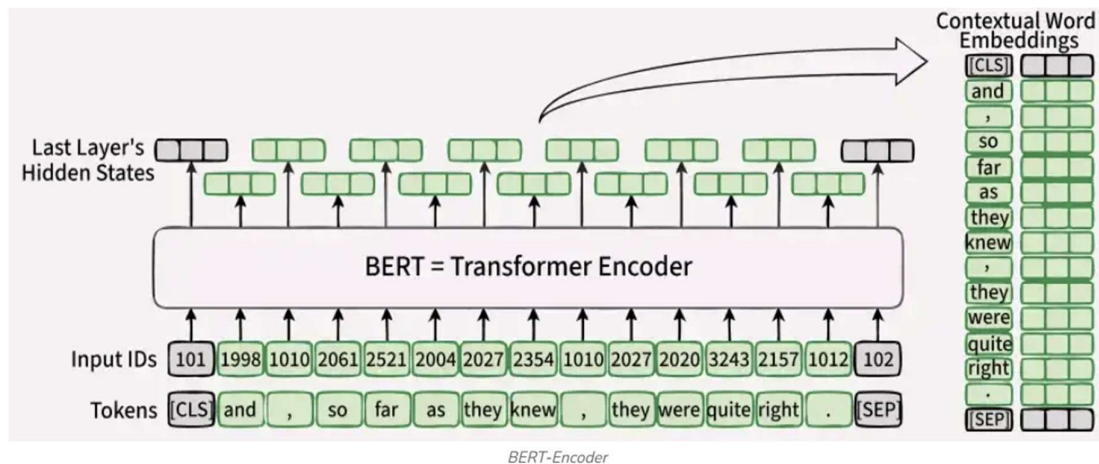
BERT Pre-training

BERT is trained on large amounts of unlabeled text to learn contextual representations of words based on their surrounding context.

- Learns embeddings that capture meaning using both left and right context
- Trained using unsupervised learning on large text datasets
- Uses tasks like predicting masked words (MLM)
- Learns relationships between sentences using Next Sentence Prediction (NSP)

Workflow of BERT

BERT uses a transformer-based encoder to process input text and generate contextualized representations for each token. Instead of predicting text sequentially like traditional models, it focuses on understanding context using its training strategies.



- Input tokens are converted into embeddings and passed through the Transformer encoder
- Produces contextual vectors for each token in the sequence
- Unlike directional models, it does not rely only on left-to-right or right-to-left processing
- Uses [Masked Language Model \(MLM\)](#) to predict missing words based on context
- Uses [Next Sentence Prediction \(NSP\)](#) to learn relationships between sentences

1. Masked Language Model (MLM)

In BERT's pre-training, some words in the input sequence are masked, and the model learns to predict these missing words using the surrounding context.

- A classification layer is added on top of the encoder outputs to predict masked words
- Output vectors are projected to the vocabulary space using the embedding matrix
- [Softmax](#) is applied to generate probability distribution over all possible words
- Loss is calculated only for masked positions, comparing predicted and actual words
- Focus on masked tokens may slow convergence compared to directional models
- However, it enables deeper contextual understanding by using both left and right context

2. Next Sentence Prediction (NSP)

Next Sentence Prediction trains BERT to understand the relationship between two sentences by predicting whether one sentence follows another.

- Uses the [CLS] token representation, passed through a classification layer
- Outputs probabilities (via Softmax) to determine if the second sentence is related
- During training, 50% of sentence pairs are actual consecutive sentences, while 50% are randomly paired
- Helps the model distinguish between logically connected and unrelated sentences
- Improves performance in tasks requiring sentence-level understanding like question answering

Combined Training of MLM and NSP

BERT is trained using both Masked Language Model (MLM) and Next Sentence Prediction (NSP) simultaneously. The model minimizes a combined loss function from both tasks, enabling it to learn deeper language understanding.

- MLM helps the model understand context within a sentence by predicting masked words

- NSP helps capture relationships between pairs of sentences
- Training both together improves understanding at both word-level and sentence-level
- Results in a more comprehensive and context-aware language model

BERT Fine-Tuning

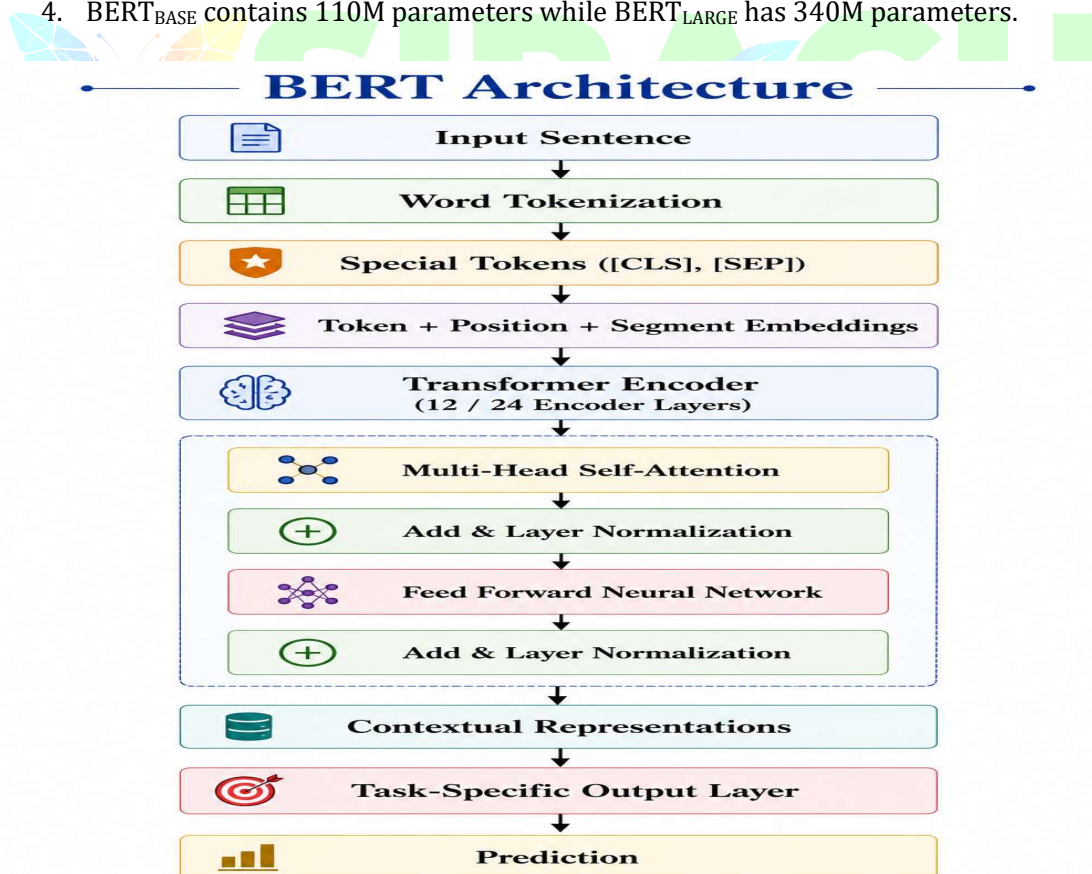
After pre-training, BERT is fine-tuned on labeled data to adapt it for specific NLP tasks. This step customizes the model's general language understanding for particular applications.

- Uses labeled datasets for task-specific training
- Adjusts model parameters based on the target task
- Applied to tasks like sentiment analysis, question answering and NER
- Requires minimal architectural changes due to its flexible design
- Enhances performance by aligning the model with task specific requirements

BERT Architecture

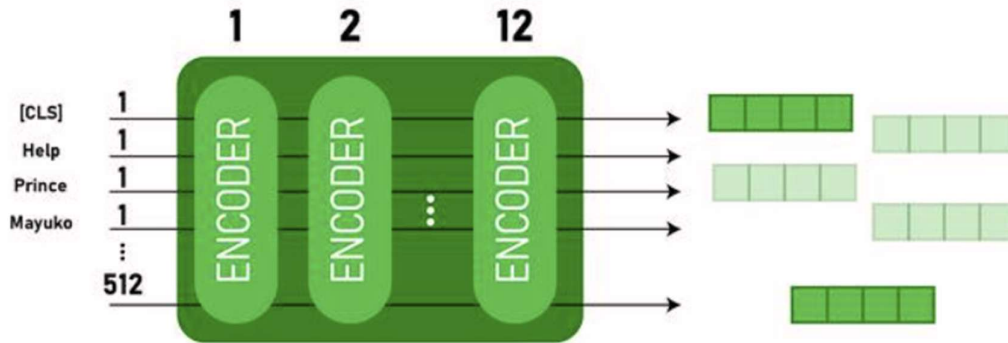
BERT uses a multilayer bidirectional Transformer encoder to understand text by capturing context from both directions. Unlike the original Transformer, which has both encoder and decoder, BERT uses only the encoder for language understanding tasks.

1. Built on a stack of Transformer encoder layers using self-attention and feed-forward networks
2. BERT_{BASE} has 12 layers in the Encoder stack while BERT_{LARGE} has 24 layers in the Encoder stack.
3. BERT architectures (BASE and LARGE) also have larger feedforward networks (768 and 1024 hidden units respectively), and more attention heads (12 and 16 respectively) than the Transformer architecture suggested in the original paper. It contains 512 hidden units and 8 attention heads.
4. BERT_{BASE} contains 110M parameters while BERT_{LARGE} has 340M parameters.



Input and Processing

- Input starts with a [CLS] token followed by the sequence of words
- Each token is converted into embeddings and passed through encoder layers
- Each layer applies self-attention and feed-forward transformations
- Outputs contextualized vectors for each token



BERT output as Embeddings

Output Usage

- The [CLS] token output is used for classification tasks
- Final embeddings can be used for tasks like classification, translation and more
- Even a simple neural layer on top of BERT can achieve strong performance

Applications of BERT in NLP

BERT can be applied to a wide range of natural language processing tasks by adding task specific layers on top of its encoder outputs.

1. Text Classification

- Used for tasks like sentiment analysis and category prediction
- The [CLS] token represents the entire input sequence
- A classification layer is added on top to make predictions

2. Question Answering

- Identifies the answer span within a given passage
- Learns to predict start and end positions of the answer
- Trained using question passage pairs

3. Named Entity Recognition (NER)

- Identifies entities such as names, organizations and locations
- Uses token-level outputs from BERT
- A classification layer predicts labels for each token

Applications of BERT

- Generates contextual embeddings for words and sentences
- Used in Named Entity Recognition (NER) to identify entities like names, locations, and organizations
- Applied in text classification tasks such as sentiment analysis, spam detection, and topic categorization
- Powers question-answering systems by identifying relevant answer spans in text
- Improves machine translation by capturing language context
- Used in text summarization to create concise representations of content
- Supports conversational AI systems like chatbots and virtual assistants
- Helps measure semantic similarity for tasks like duplicate detection and information retrieval

GENERATIVE PRE-TRAINED TRANSFORMER (GPT)

Generative Pre-trained Transformer (GPT) is a deep learning language model developed to understand and generate human-like text. It is based on the Transformer architecture, specifically the Transformer Decoder, and is pre-trained on massive collections of text obtained from books, articles, websites, and other textual sources. During training, GPT learns the statistical patterns, relationships, and structures present in natural language, enabling it to generate meaningful and contextually relevant responses.

Unlike traditional NLP models that are designed for a single task, GPT is a general-purpose language model. Once pre-trained, the same model can perform a wide variety of language-related tasks such as text generation, summarization, question answering, translation, code generation, and conversational AI with little or no task-specific modification.

Characteristics of GPT

1. Can Read and Understand Human Language

GPT is designed to process and interpret text written in natural language. During pre-training, it is exposed to billions of words from diverse text sources, allowing it to learn vocabulary, sentence structures, grammar, and the relationships between words. By analysing the context of previously seen words, GPT develops an understanding of the meaning conveyed in a sentence and can interpret user inputs effectively.

2. Generate Meaningful and Natural-Sounding Text

One of the primary capabilities of GPT is generating text that closely resembles human writing. Based on the input provided, the model predicts the most appropriate sequence of words to produce coherent, grammatically correct, and contextually meaningful responses. Since it has learned language patterns from extensive training data, the generated text often maintains logical flow and consistency throughout the response.

3. Trained on Large Datasets to Learn Language Patterns

GPT is pre-trained using extremely large text datasets containing information from various domains and writing styles. Rather than memorizing sentences, the model learns the statistical relationships between words, phrases, and sentences. Through this large-scale pre-training process, GPT acquires knowledge about language structure, word usage, context, and common linguistic patterns, which enables it to respond appropriately to a wide range of inputs.

4. Can Write, Summarize, Answer Questions, and Even Help in Coding

A single GPT model can perform multiple natural language processing tasks without requiring separate models for each task. Depending on the user's prompt, GPT can generate articles, compose emails, summarize lengthy documents, answer factual or conceptual questions, explain technical concepts, generate programming code, assist in debugging, and provide code explanations. This versatility makes GPT a powerful general-purpose language model capable of handling diverse language-related applications.

5. Understands How Words Relate to Each Other to Give Relevant Responses

GPT learns the contextual relationships between words during its training process. Instead of treating each word independently, it considers the surrounding words to determine the intended meaning of the input. By understanding these contextual relationships, GPT generates responses that are relevant, coherent, and appropriate for the given context. This ability enables the model to maintain continuity across sentences and produce responses that align closely with the user's input.

Working of GPT

GPT (Generative Pre-trained Transformer) works by learning patterns in language through a two-stage process: Pre-training and Fine-tuning. The model is built on the Transformer architecture, which enables it to process text efficiently and understand the relationships between words in a sentence. Instead of processing one word at a time, GPT processes multiple words simultaneously and predicts the next word based on the context of the previous words.

1. Pre-training

The first stage in the working of GPT is Pre-training.

Pre-training is the process in which GPT is trained on vast amounts of text data collected from multiple sources. During this stage, the model is not trained for any specific application. Instead, it learns the general structure and characteristics of human language.

The objective of pre-training is to build a language model that understands how words are used together in different contexts.

During pre-training, GPT learns:

- Language patterns
- Grammar
- Facts
- Some reasoning abilities

Learning Language Patterns

During pre-training, GPT reads a massive collection of text and learns the statistical relationships between words and sentences.

Rather than memorizing individual sentences, the model learns how words typically occur together and how they form meaningful expressions.

By repeatedly observing these patterns, GPT develops an understanding of natural language structure, enabling it to generate coherent and contextually appropriate text.

Learning Grammar

Grammar defines the rules governing sentence formation.

During pre-training, GPT learns grammatical structures by analysing countless examples of correctly written text.

Through repeated exposure, the model learns how words combine to form grammatically correct sentences and how different sentence structures are used in natural language.

Learning Facts

The large text corpus used during pre-training contains information from numerous domains. By processing this information, GPT learns factual knowledge that appears in the training data.

This knowledge enables the model to answer factual questions, explain concepts, and generate informative responses based on the information it has learned during training.

Learning Some Reasoning Abilities

During pre-training, GPT also develops certain reasoning capabilities by learning relationships between concepts that frequently appear together in text.

Although the model is not explicitly taught logical reasoning rules, repeated exposure to diverse language patterns enables it to identify common relationships and generate responses that appear logically consistent in many situations.

These reasoning abilities emerge naturally as a result of large-scale language modelling during pre-training.

Output of Pre-training

After completing the pre-training stage, GPT becomes a pre-trained language model.

At this point, the model has acquired a broad understanding of language and can generate coherent text, but it has not yet been specifically adapted for particular applications or user requirements.

2. Fine-tuning

The second stage is Fine-tuning.

Fine-tuning is the process of further training the already pre-trained GPT model using specific datasets together with human feedback.

The objective of fine-tuning is to adapt the model so that its responses align more closely with the desired behaviour for particular applications.

Instead of learning general language patterns, the model now learns how to produce responses that are more accurate, helpful, and appropriate for the intended use.

Training on Specific Datasets

During fine-tuning, GPT is trained using datasets prepared for particular domains or tasks.

These datasets help the model adjust its knowledge to better suit the required application while retaining the general language understanding acquired during pre-training.

This additional training enables GPT to generate responses that are more relevant for the intended task.

Human Feedback

An important part of fine-tuning is the use of human feedback.

Human reviewers evaluate the responses produced by the model and provide guidance regarding which responses are more appropriate or useful.

The model uses this feedback during training to improve the quality of its future responses.

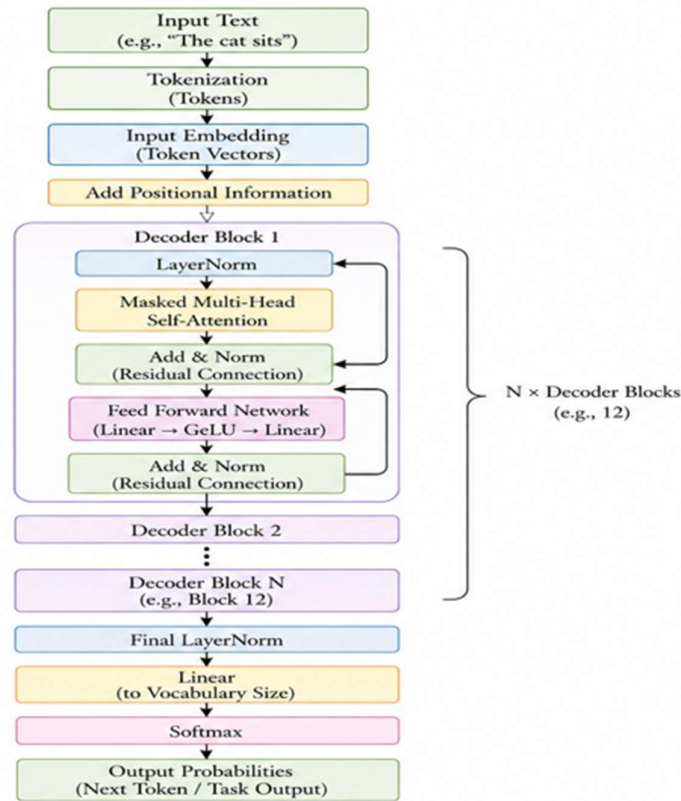
As a result, GPT becomes better aligned with human expectations regarding correctness, relevance, and helpfulness.

Aligning Responses with Desired Outputs

The primary goal of fine-tuning is to ensure that the responses generated by GPT closely match the expected or desired outputs.

By combining task-specific datasets with human feedback, the model learns to produce responses that are more suitable for practical applications while maintaining the language knowledge acquired during pre-training.

This alignment improves the overall quality, consistency, and usefulness of the generated text.



1. Input Embedding

- **Input:** The text is first broken into tokens (words or subwords).
- **Embedding:** Each token is converted into a dense vector representation.

2. Positional Information

Since the model does not understand order by default, positional information is added to the embeddings to preserve sequence order.

3. Decoder Blocks

- **LayerNorm:** Each block starts with layer normalization.
- **Masked Multi-Head Self-Attention:** The model attends only to previous tokens using multiple attention heads.
- **Add & Norm:** The attention output is added to the input (residual connection) and normalized.
- **Feed Forward:** A feed-forward network (Linear -> GeLU -> Linear) processes the data further.
- **Add & Norm:** Again, the output is added back and normalized.

4. Layer Stack

Multiple decoder blocks are stacked (e.g., 12 blocks) to capture deeper patterns and relationships in the text.

5. Final Layers

- **LayerNorm:** A final normalization is applied.
- **Linear:** The output is mapped to the vocabulary size.
- **Softmax:** Probabilities are generated for predicting the next token or task output.

Advantages

- Capable of handling diverse tasks with minimal adaptation.
- Deep learning enables comprehension of complex text..
- Performance improves with data size and model parameters.
- Learns new tasks from limited examples.
- Generates novel and coherent content.

T5 (TEXT-TO-TEXT TRANSFER TRANSFORMER)

T5 (Text-to-Text Transfer Transformer) is a transformer-based language model developed by Google Research. It treats every NLP task as a text-to-text problem, allowing the same model to perform a wide range of language tasks through a unified framework.

- Uses an encoder-decoder transformer architecture.
- Handles multiple NLP tasks using task-specific text prompts.
- Generates text-based outputs for different language tasks.
- Commonly used for translation, summarization and question answering.

Working of T5

T5 is built on a Transformer Encoder-Decoder architecture, similar to sequence-to-sequence Transformer models. The encoder is responsible for understanding the input text and generating contextual representations, while the decoder generates the corresponding output text based on these representations. The complete working of T5 consists of four major stages.

1. Task Formulation as Text-to-Text

The fundamental principle of T5 is to convert every NLP task into a **text-to-text format**. Instead of treating different natural language processing problems separately, T5 represents every task using a textual input and produces a textual output.

In traditional NLP systems, different tasks often require different output formats. For example, text classification may produce class labels, translation generates another language, while summarization generates a shorter version of the input. T5 eliminates this difference by expressing every task as a text generation problem.

The input to the model always consists of a textual description of the task together with the input text. The output is also generated entirely as text.

This unified formulation allows the same model architecture to perform multiple NLP tasks without changing the underlying network structure.

Therefore, regardless of the application, T5 processes the task using the same text-to-text framework.

2. Encoding the Input

Once the task has been represented in text form, the input text is tokenized using SentencePiece.

SentencePiece converts the input sentence into smaller subword units or tokens that can be processed by the model. These tokens are then converted into numerical representations before being supplied to the encoder.

After tokenization, the sequence enters the Transformer Encoder.

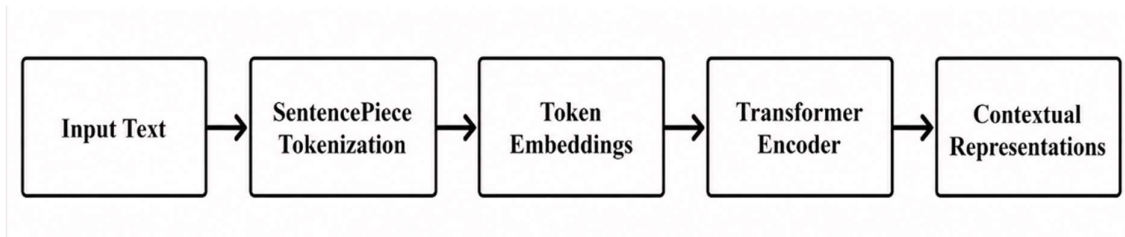
The encoder processes all input tokens simultaneously and generates contextual representations for every token in the sequence.

Instead of processing words independently, the encoder analyses the relationships between all tokens present in the input. Through multiple encoder layers, the contextual information of each token is continuously refined.

As the input passes through the encoder, every token representation gradually incorporates information from the surrounding tokens, resulting in rich contextual representations that capture the overall meaning of the input sequence.

The final output of the encoder is a set of contextual representations, which serve as the input for the decoder.

The overall encoding process can be represented as:



3. Decoding the Output

After the encoder generates the contextual representations, these representations are passed to the Transformer Decoder.

The decoder uses the encoded information to generate the required output text.

Unlike the encoder, which processes the complete input simultaneously, the decoder generates the output in an autoregressive manner.

In autoregressive generation, the decoder predicts one output token at a time. After generating a token, it uses the previously generated output tokens together with the encoded input representation to predict the next token.

This process continues sequentially until the complete output sentence has been generated.

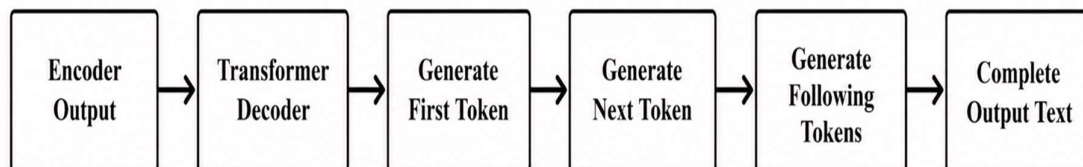
At each decoding step, the decoder refers to:

- the contextual representations produced by the encoder, and
- the output tokens generated so far.

Using this information, it predicts the next most appropriate token in the output sequence.

The decoding process continues repeatedly until the model determines that the output sequence is complete.

The decoding workflow can be represented as:



4. Training the Model

Before T5 can perform various NLP tasks, it undergoes a pre-training stage followed by fine-tuning.

During pre-training, T5 uses a denoising objective.

In this approach, portions of the original text are intentionally masked or removed from the input sequence. The model is then trained to reconstruct the missing portions using the remaining context.

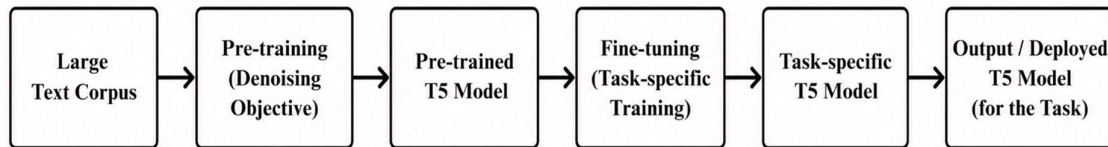
By repeatedly learning to recover the missing text, T5 develops a strong understanding of language structure, contextual relationships, and sentence semantics.

The denoising objective enables the encoder and decoder to learn meaningful representations of natural language from large amounts of text data.

After completing pre-training, the model undergoes fine-tuning for various NLP tasks.

During fine-tuning, the already pre-trained model is trained further using task-specific datasets. This allows the model to adapt its general language understanding to perform particular applications while retaining the knowledge acquired during pre-training.

Thus, the complete training process consists of two stages:



Applications

- Generates natural and context-aware responses for chatbots and virtual assistants.
- Produces concise summaries of long documents and articles.
- Translates text accurately between multiple languages.
- Answers user questions by understanding the context of the input.

Advantages

- Uses a single framework to perform multiple NLP tasks.
- Produces high quality text for generation and understanding tasks.
- Easily adapts to different tasks through task specific prompts.
- Leverages transfer learning to achieve strong performance with fine tuning.

LLAMA (LARGE LANGUAGE MODEL META AI)

LLaMA (Large Language Model Meta AI) is a family of advanced language models developed by Meta (formerly Facebook) that can understand and generate human-like text, making them highly valuable for tasks in natural language processing (NLP), conversational AI, text generation and other AI-driven language applications.

Key Features of LLaMA

- Strong performance in NLP tasks like generation, translation and summarization.
- Transformer-based architecture with self-attention for context understanding.
- Supports fine-tuning across different environments.
- Adaptable for tasks like chatbots and content generation.
- Available in multiple sizes (7B–65B) for different needs.
- Accessible for research and development.

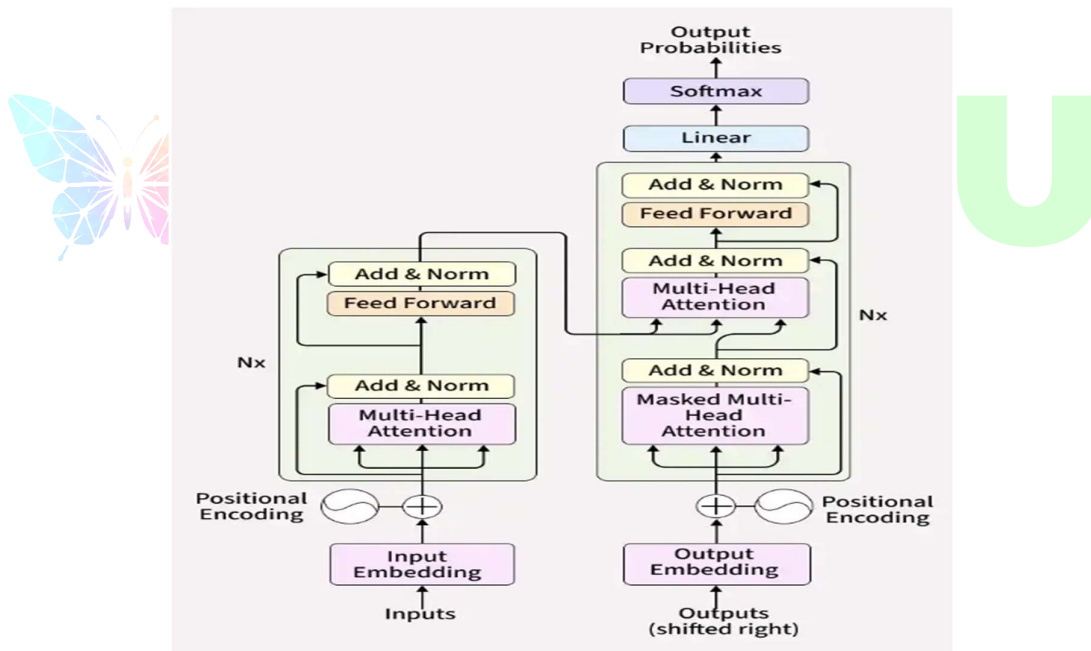
Architecture

- **Transformer-Based:** LLaMA uses the Transformer architecture, which processes sequences in parallel and captures long-range dependencies using self-attention mechanisms.
- **Stacked Transformer Blocks:** The model consists of multiple layers, each including a multi-head self-attention mechanism followed by a feedforward neural network to extract complex patterns in text.
- **Multiple Model Sizes:** Available in different scales to balance performance and hardware requirements: LLaMA-7B, LLaMA-13B, LLaMA-30B and LLaMA-65B parameters.
- **Positional Encoding:** Uses positional information to understand the order of words in a sentence, ensuring proper context comprehension.

- **Parallel Processing:** Self-attention allows the model to analyze all words in a sequence simultaneously, making training and inference more efficient.
- **Contextual Understanding:** Each layer refines the model's understanding of text, enabling LLaMA to generate coherent and contextually accurate outputs.
- **Rotary Positional Embeddings (RoPE):** LLaMA uses RoPE instead of traditional positional encoding, allowing better handling of long sequences by encoding relative positions.
- **RMSNorm:** Instead of Layer Normalization, LLaMA uses Root Mean Square Normalization (RMSNorm), which improves training stability and efficiency.

Applications

- **Conversational AI:** Powers chatbots and virtual assistants capable of natural, context-aware conversations, improving customer engagement and support.
- **Content Creation:** Automates writing for blogs, social media, product descriptions and marketing materials, saving time while maintaining quality.
- **Machine Translation:** Enables accurate multilingual translation for documents, reports and communications across different languages.
- **Sentiment Analysis:** Analyzes text to determine sentiment, helping brands monitor customer feedback, social media and product reviews.
- **Text Summarization:** Condenses long documents or articles into concise summaries, making large volumes of information easier to digest.



Advantages

- **High Performance:** Achieves state-of-the-art results across NLP tasks like text generation, summarization, translation and sentiment analysis.
- **Scalable and Flexible:** Can be trained and fine-tuned on a range of hardware setups, from high-end GPUs to accessible computing environments.
- **Versatile Applications:** Useful for chatbots, content creation, translation, sentiment analysis and more across multiple industries.
- **Open-Source Friendly:** Meta provides access to models and documentation, promoting research, collaboration and innovation.

- **Context-Aware Understanding:** Uses Transformer-based architecture with self-attention mechanisms to maintain coherence and context in generated text.

Limitations

- **Resource Intensive:** Large models require significant computing power and memory for training and inference.
- **Potential Bias:** Like other language models, LLaMA can reflect biases present in training data, affecting fairness and neutrality.
- **Interpretability Challenges:** Understanding why the model generates a specific output can be difficult, limiting transparency.
- **Ethical Concerns:** Misuse in generating fake content, misinformation or spam is possible if not carefully monitored.
- **Fine-Tuning Requirements:** Domain-specific tasks may require additional fine-tuning to achieve optimal performance.

TYPES OF TOKENIZATION TECHNIQUES

Tokenization is the process of breaking text into smaller parts called tokens, such as words, sentences, or characters. Different tokenization techniques are used in Natural Language Processing (NLP) depending on the task.

- Converts raw text into a format that AI models can understand
- Helps in analyzing and processing text efficiently
- Useful for improving the accuracy of NLP models

1. Word Tokenization

Word Tokenization is the simplest and most commonly used tokenization technique. In this approach, the input text is divided into individual words, where each word is considered as a separate token. The splitting process is generally performed using spaces that separate words in a sentence.

During word tokenization, the model treats every word as a single unit and does not divide it into smaller components. Each token corresponds directly to one complete word present in the input text.

Since the words are not broken into smaller parts, this method is straightforward and easy to implement. It is widely used in basic text processing applications where words themselves provide sufficient information.

Characteristics

- Splits the input text based on spaces.
- Each complete word is treated as one individual token.
- Words are not divided into smaller meaningful units.

Example

Input

Machine learning is powerful

Output

["Machine", "learning", "is", "powerful"]

Here, the sentence is divided into four individual word tokens.

Advantages

- Simple and easy to implement.
- Computationally efficient for basic text processing tasks.

Disadvantages

- Cannot effectively handle unseen or unknown words.
- Does not capture the internal structure or context present within individual words.

2. Sentence Tokenization

Sentence Tokenization is the process of dividing a large block of text into individual sentences. Instead of splitting text into words, this method identifies sentence boundaries and separates each sentence as an independent unit.

Sentence boundaries are usually identified using punctuation marks such as the full stop (.), question mark (?), and exclamation mark (!). Each sentence obtained after tokenization can then be processed independently for further analysis.

Sentence tokenization is particularly useful when dealing with lengthy documents, as it organizes the text into smaller and more manageable units.

Characteristics

- Divides text into individual sentences.
- Uses punctuation marks such as ., ?, and ! to identify sentence boundaries.
- Converts large documents into smaller sentence units.

Example

Input

AI is transforming industries. It is used everywhere.

Output

```
[
  "AI is transforming industries.",
  "It is used everywhere."
]
```

The input paragraph is divided into two separate sentence tokens.

Advantages

- Organizes long text into smaller and structured sentences.
- Useful for analysing relationships between different sentences.

Disadvantages

- May incorrectly identify sentence boundaries when punctuation is used in complex forms.
- Sentence boundary detection rules may vary across different languages.

3. Subword Tokenization

Subword Tokenization divides words into smaller meaningful units called subwords instead of treating the entire word as a single token.

Many natural language processing models encounter words that were not present during training. Instead of treating these words as completely unknown, subword tokenization divides them into smaller parts that may already exist in the vocabulary.

This allows the model to process rare, complex, or previously unseen words more effectively while maintaining a manageable vocabulary size.

Subword tokenization provides a balance between word-level tokenization and character-level tokenization.

Characteristics

- Splits long or complex words into smaller meaningful pieces.
- Enables the model to process unknown words using known subwords.
- Combines the advantages of both word-level and character-level representations.

Example

Input

playing

Output

["play", "ing"]

The word is divided into two meaningful subword units.

Advantages

- Effectively handles unseen or unknown words.
- Reduces the overall vocabulary size while preserving meaningful word components.

Disadvantages

- More computationally complex than simple word tokenization.
- Some words may be divided into subwords that appear less natural.

4. Character Tokenization

Character Tokenization divides the input text into individual characters, where every character is considered a separate token.

Unlike word-based approaches, this method does not rely on words or vocabulary. Every letter, digit, or symbol is processed independently.

Since all words are represented as sequences of characters, the method naturally handles unknown or newly encountered words without requiring them to exist in the vocabulary.

Character tokenization follows the same procedure regardless of the language being processed.

Characteristics

- Splits every word into its individual characters.
- Does not depend on a predefined vocabulary.
- Processes text at the character level.

Example

Input

Data

Output

["D", "a", "t", "a"]

Each character becomes an independent token.

Advantages

- Does not encounter problems with unknown or rare words.
- Can be applied consistently across different languages.

Disadvantages

- Produces significantly longer token sequences.
- Longer sequences require more processing time and computational resources.

5. Byte Pair Encoding (BPE)

Byte Pair Encoding (BPE) is a subword tokenization technique that constructs subword units by repeatedly merging the most frequently occurring pairs of characters or existing subwords.

Instead of storing every complete word in the vocabulary, BPE gradually builds larger subword units from frequently occurring character sequences. As a result, commonly occurring words remain as larger units, while uncommon or rare words are represented using smaller subword components.

This approach reduces the vocabulary size while preserving meaningful subword information, making it highly suitable for modern Natural Language Processing models.

Characteristics

- Repeatedly merges the most frequently occurring character pairs or subword pairs.
- Creates meaningful subword units from frequently occurring patterns.
- Produces a compact vocabulary while maintaining language representation.

Example

Input

lower

Output

["low", "er"]

The original word is divided into two frequently occurring subword units.

Advantages

- Handles rare and previously unseen words effectively.
- Produces a balanced vocabulary size by representing words as reusable subword units.

Disadvantages

- Requires training on large text datasets to identify frequent character or subword pairs.
- The merging process is more complex than basic tokenization methods and requires additional implementation effort.

EMBEDDINGS AND VECTOR REPRESENTATIONS

Humans naturally understand language by interpreting words, grammar, context, and meaning. For example, when reading the sentence:

A computer, however, does not possess any inherent understanding of language.

Internally, computers operate only on binary digits (0s and 1s) and perform mathematical computations on numerical data.

A vector representation is a numerical description of a word using multiple numerical values instead of a single number.

Each value in the vector represents one learned numerical feature.

These features are not manually defined.

Instead, they are automatically learned during model training.

Mathematically,

a word vector is represented as

$$\mathbf{w} = [w_1, w_2, w_3, \dots, w_d]$$

where

- d is the embedding dimension,
- each w_i is a real-valued number.

An embedding is a dense vector representation that maps textual information into a continuous numerical space while preserving semantic relationships between words.

Instead of representing words as isolated symbols, embeddings place semantically related words close together within a multidimensional vector space.

An embedding can represent:

- a word,
- a phrase,
- a sentence,
- a paragraph,
- or an entire document.

The general mapping performed by an embedding model is

$$f: T \rightarrow \mathbb{R}^d$$

where

- T represents text,
- d represents the embedding dimension.

SCALING LAWS IN LARGE LANGUAGE MODELS

Scaling Laws are empirical principles that describe how the performance of Large Language Models (LLMs) improves as three fundamental factors are increased:

1. Model Size (Number of Parameters)
2. Training Dataset Size (Number of Tokens)
3. Training Compute (Computational Resources)

These laws provide a mathematical understanding of how language models behave when scaled from millions to billions or even trillions of parameters. Rather than relying solely on trial-and-error experimentation, scaling laws enable researchers to predict model performance before training, estimate the required computational resources, determine the optimal balance between model size and dataset size, and design more efficient large-scale language models.

The concept of scaling laws became prominent through empirical studies conducted by OpenAI and other research organizations, demonstrating that language model performance follows predictable mathematical relationships as the scale of training increases.

Unlike architectural improvements, scaling laws focus on the effect of increasing scale while keeping the underlying Transformer architecture largely unchanged.

Training a modern Large Language Model requires enormous computational resources.

For example, increasing the number of parameters from hundreds of millions to hundreds of billions significantly increases:

- Training time
- GPU memory requirements
- Storage requirements
- Energy consumption
- Financial cost

Without theoretical guidance, determining the appropriate model size or dataset size becomes extremely expensive.

Components of Scaling Laws

The performance of an LLM depends primarily on three variables.

1. Model Size

Model size refers to the total number of trainable parameters.

Parameters include:

- Token embedding matrices
- Position embeddings
- Query projection matrices
- Key projection matrices
- Value projection matrices
- Attention output projections
- Feed-forward network weights
- Layer normalization parameters
- Output projection layers

If N represents the number of trainable parameters, then

$$N = \sum_{i=1}^L \text{Parameters in Layer } i$$

where

$$L$$

denotes the total number of Transformer layers.

Typical model sizes include:

Model	Approximate Parameters
GPT-2 Small	117 Million
GPT-2 XL	1.5 Billion
GPT-3	175 Billion
GPT-4	Not publicly disclosed
LLaMA 2	7B–70B
LLaMA 3	8B–70B+

As the number of parameters increases, the representational capacity of the model also increases.

Larger models can learn:

- richer semantic relationships,
- more complex syntactic structures,
- broader factual knowledge,
- improved contextual reasoning,
- and more generalized language representations.

2. Training Dataset Size

Training data refers to the total number of tokens used during pre-training.

A token may represent:

- a complete word,
- a subword,
- punctuation,
- or a special token.

If

$$D$$

denotes the number of training tokens,
then

$$D = \text{Total Training Tokens}$$

Training datasets often contain hundreds of billions or trillions of tokens collected from:

- books,
- research articles,
- web pages,
- code repositories,
- encyclopedias,
- technical documents,
- multilingual corpora.

Increasing training data enables the model to observe:

- more vocabulary,
- more language structures,
- additional factual information,
- broader contextual relationships.

However, increasing data alone cannot fully improve performance if the model capacity remains insufficient.

3. Training Compute

Training compute represents the total computational effort required to optimize the model. It depends on:

- model parameters,
- dataset size,
- training iterations,
- floating-point operations.

Training compute is commonly measured using:

- FLOPs (Floating Point Operations)
- GPU hours
- TPU hours

A simplified approximation is

$$C \propto N \times D$$

where

- C denotes compute,
- N represents model parameters,
- D represents training tokens.

This relationship indicates that increasing either model size or training data substantially increases computational cost.

Chinchilla Scaling Law

Although the OpenAI Scaling Laws demonstrated that increasing model size improves performance, later research conducted by DeepMind observed an important limitation. Many early language models, including GPT-3, were found to be over-parameterized relative to the amount of training data.

the models contained an extremely large number of parameters but were trained using comparatively fewer tokens than would be ideal.

As a result, many parameters were not fully utilized during training. This observation led to the development of the Chinchilla Scaling Law.

Main Principle

The Chinchilla Scaling Law states that, for a fixed computational budget, better performance is achieved by balancing model size and training data rather than simply increasing the number of parameters.

Instead of continuously increasing model parameters, the computational budget should be distributed between

- model parameters
- training tokens

in an optimal proportion.

Chinchilla Strategy

Although both approaches require approximately the same computational cost, the second approach produces

- lower training loss,
- better generalization,
- higher compute efficiency.

Key Insights from Chinchilla Scaling Laws

1. **Optimal Allocation of Compute Resources:** Instead of focusing solely on increasing model size, Chinchilla Scaling Laws emphasize the importance of allocating compute resources across both model size and training data. For a fixed compute budget, the optimal strategy is to use fewer parameters but more training data. This approach ensures that the model is neither overparameterized nor undertrained, maximizing its learning potential.
2. **Data Efficiency:** Larger datasets allow models to generalize better and reduce overfitting. By prioritizing training data over model size, developers can achieve higher accuracy without requiring exponentially larger models.
3. **Energy and Cost Efficiency:** Training massive models consumes significant energy and financial resources. Chinchilla's findings suggest that smaller, well-trained models can deliver equivalent or superior performance at a fraction of the cost and environmental impact.
4. **Scalability Across Domains:** While Chinchilla Scaling Laws were initially validated in the context of language models, their principles are likely applicable to other domains, including computer vision, reinforcement learning, and multimodal AI systems.

OpenAI Scaling Laws

The OpenAI Scaling Laws describe the empirical relationship between the performance of a Large Language Model (LLM) and three fundamental scaling factors:

1. Model Size (Number of Parameters)
2. Training Dataset Size (Number of Tokens)
3. Training Compute (Computational Budget)

These scaling laws were introduced through extensive empirical experiments conducted by OpenAI researchers, where hundreds of Transformer-based language models of different sizes were trained under varying computational budgets. The objective was to determine how model performance changes as these three variables are systematically increased.

One of the most significant observations was that language model performance follows predictable mathematical trends rather than improving randomly. Specifically, the prediction error or training loss decreases according to a power-law relationship as the model size, dataset size, and computational resources increase.

This discovery fundamentally changed the design philosophy of modern Large Language Models. Instead of relying solely on architectural modifications, researchers realized that simply scaling existing Transformer architectures in a balanced manner could produce substantial improvements in language understanding and generation.

Scaling with Model Parameters

The first scaling law states that increasing the number of trainable parameters generally improves model performance.

Suppose N represents the total number of trainable parameters.

The empirical relationship can be expressed as

$$L(N) = AN^{-\alpha} + B$$

where

- $L(N)$ = Training Loss
- N = Number of Parameters
- A, B = Empirical Constants
- α = Scaling Exponent

As the number of parameters increases,

- language representations become richer,
- contextual understanding improves,
- reasoning capabilities increase,
- prediction error decreases.

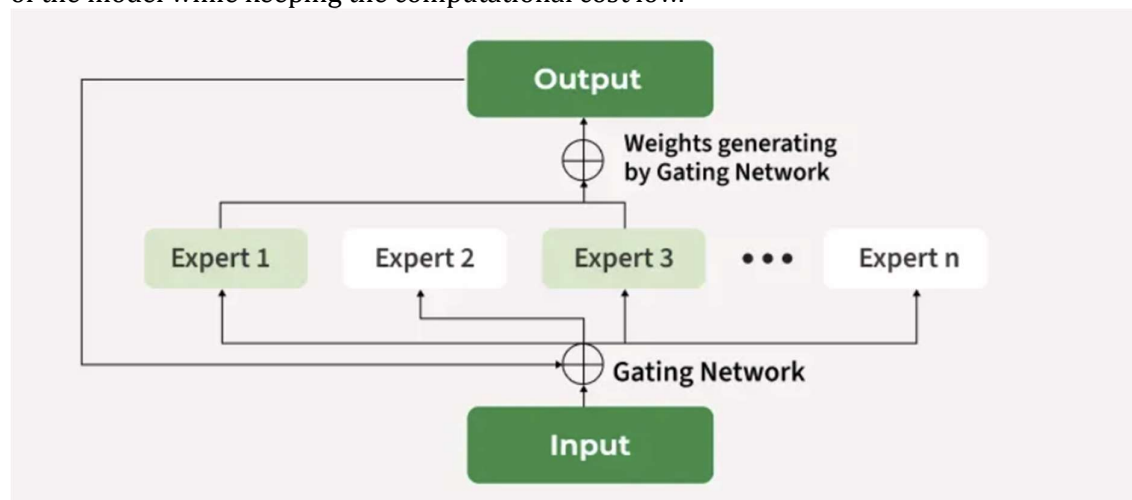
However, the improvement does not occur linearly.

Initially, increasing model parameters produces significant performance gains. As the model becomes larger, each additional increase produces progressively smaller improvements.

This phenomenon is known as diminishing returns.

MIXTURE OF EXPERTS (MoE) ARCHITECTURES

Mixture of experts (MoE) is a machine learning approach that divides a machine learning model into separate sub networks or experts where each of these experts specialize in a subset of the input data to jointly perform a task. This approach helps to increase efficiency of the model while keeping the computational cost low.

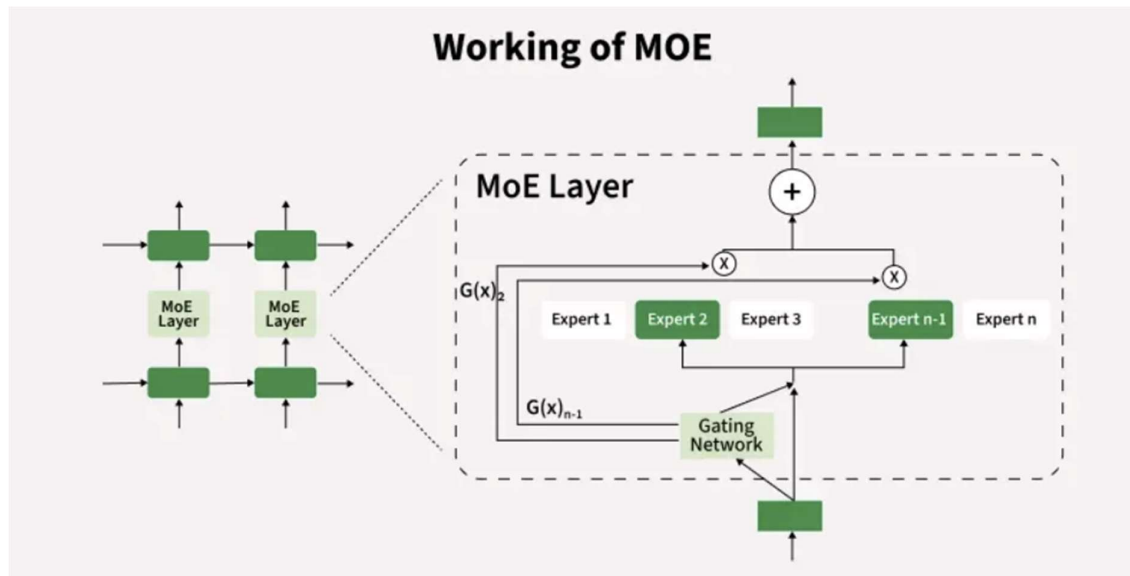


- **Input:** This is the data you want the machine learning model to handle.
- **Experts:** These are smaller sub networks of the model where each is trained to be really good at a specific part of the problem.
- **Gating network:** This is like a manager who decides which expert or sub network is best suited for each part of the problem.
- **Output:** This is the final solution that the machine learning model produces after the experts have done their work.

Working of MoE

MOE works in two phases:

1. Training phase
2. Inference phase



Training phase

1. Training the Experts

- Each expert is trained on a specific part of the data or a specific problem our model wants to solve.
- Each expert learns by itself using the usual training method and try to reduce its own errors.

2. Training the Gating Network

- The gating network is like a traffic controller which looks at the input and decides which expert should handle it.
- While the experts are being trained, the gating network also learns to assign probabilities to each expert who is most likely to give the best output.
- The gating network computes scores $G(x)_1, \dots, G(x)_n$
- The gating network is trained to make better decisions over time by checking how well the chosen experts perform.

3. Joint Training

- At this stage the experts and the gating network are trained together
- They work as a team as the model tries to reduce the total error from both the experts and the gating network and updates are made to improve the whole system not just individual parts.
- This makes sure that the gating network picks the right experts and that the experts give good outputs.

Inference phase

1. Input routing

- When a new input comes in the gating network decides which experts should handle it.
- It looks at the input and gives probabilities scores to each expert and based on these scores it sends the input to the most suitable experts which helps the system use the right expert for each task.

2. Expert selection

- Instead of using all experts MOE models picks only one based on who got the highest probabilities scores from the gating network.
- This saves time and computing power while using the experts that are best suited for that task.

3. Output Combination

- After the chosen experts do their work, their answers are combined into one final result.
- This is usually done using weighted averaging where the expert with the highest probability score has more influence.
- The goal is to get a more accurate and balanced answer by using the strengths of each expert.

Applications

1. **Natural language processing (NLP):** In traditional models the entire model is used every time you give it input even if it is not needed which takes a lot of time and computing power. Whereas in an MoE model experts are used based on what the input needs. This is called sparsity and it helps the model work faster and use less power without losing accuracy.
2. **Computer vision:** MOE models do not look at the whole image at once they split the image into small patches and these patches go through a gating network which decides which expert should handle each patch. This helps the model be more accurate and efficient.
3. **Recommendation systems:** MOE's are popular in recommendation systems because they can break a large problem into smaller tasks, each handled by a simple expert which makes training faster and works well for large scale systems.

Advantages

1. **Flexibility:** The diversity of tasks between experts make MOE models highly flexible.
2. **Fault tolerance:** MoE's use 'divide and conquer' approach where tasks are executed separately which enhances the model's resilience to failures.
3. **Scalability:** MOE's decompose complex problems into smaller and more manageable tasks which helps MoE models handle increasingly complicated inputs.

Disadvantages

1. **Complexity in training phase:** training MOE models can be tricky because it requires coordination between the experts and the gating network which is hard to achieve.
2. **Low Inference efficiency:** The gating network needs to run for each input to determine the right experts which adds extra computation and running multiple experts in parallel can be challenging in environments with limited computational resources.
3. **Increased model size:** Storing multiple expert networks and the gating network increases the overall storage of the model and deploying such models is harder due to their size and comp

EMERGENT CAPABILITIES IN LARGE LANGUAGE MODELS (LLMs)

Emergent capabilities are advanced abilities that appear in Large Language Models (LLMs) when the model reaches a sufficiently large scale in terms of parameters, training data, and computational resources. These capabilities are often not explicitly programmed and may not be present in smaller versions of the same model. In other words, as models become larger, they begin to perform tasks that were previously impossible or very difficult, demonstrating behaviors that seem to "emerge" naturally from large-scale learning.

Emergent Capability: A behavior or skill that appears suddenly in a large model but is absent or weak in smaller models, even though both models are trained using similar methods.

Large Language Models learn patterns from enormous amounts of text data. As model size increases:

- ✚ More complex patterns can be captured.
- ✚ Long-range relationships between words are better understood.
- ✚ Abstract reasoning abilities improve.
- ✚ Knowledge from different domains can be combined.

The interaction between billions of parameters enables the model to develop new problem-solving abilities that were not directly taught during training.

Characteristics of Emergent Capabilities

- 1. Sudden Appearance:** Performance may remain low for smaller models and then improve dramatically once a certain scale is reached.
- 2. Not Explicitly Programmed:** Developers do not manually create these skills.
- 3. Scale Dependent:** Usually observed in very large models trained on extensive datasets.
- 4. Generalizable:** Capabilities can be applied to many different tasks.
- 5. Unexpected:** Some behaviors are not anticipated during model design.

Major Emergent Capabilities of LLMs

- 1. In-Context Learning:** The ability of a model to learn a task from examples provided within the prompt without updating model parameters.

Example

Prompt:

English → French
Dog → Chien
Cat → Chat

Oiseau

The model learns the pattern from the examples and applies it to a new input.

Advantages

- No retraining required.
- Adapts quickly to new tasks.
- Reduces computational cost.

Applications

- Translation
- Classification
- Question Answering

- Text Summarization

2. Zero-Shot Learning: The ability to perform a task without seeing any examples during prompting.

Example

Prompt:

Classify the sentiment:

"I enjoyed the movie."

Positive

Even without examples, the model understands the task.

Benefits

- ✚ Saves training effort.
- ✚ Enables flexible deployment.
- ✚ Handles unseen tasks.

Applications

- ✚ Sentiment Analysis
- ✚ Text Classification
- ✚ Intent Detection

3. Few-Shot Learning: The ability to learn from a small number of examples included in the prompt.

Example

Prompt:

Positive: I love this product.

Negative: This is terrible.

Positive: Excellent service.

Classify: "The food was amazing."

Positive

Advantages

- Requires only a few examples.
- Works for many tasks without fine-tuning.
- Useful when labeled data is limited.

4. Chain-of-Thought (CoT) Reasoning: The ability to solve complex problems through intermediate reasoning steps.

Example

Question:

A shop sells 3 pens for ₹30.

How much will 9 pens cost?

₹90

3 pens = ₹30

1 pen = ₹10

9 pens = ₹90s = ₹90

Importance

Allows models to:

- Solve mathematical problems.
- Perform logical reasoning.
- Handle multi-step decision making.

Applications

- Mathematics
- Coding
- Scientific problem solving
- Logical puzzles

5. Code Generation: The ability to generate computer programs from natural language descriptions.

Example

Prompt:



Write a C++ program to find the factorial of a number.

```
#include <iostream>

using namespace std;

int main() {

    int n;

    long long fact = 1;

    for(int i = 1; i <= n; i++)

        fact *= i;

    cout << fact;
```

Applications

- Software development
- Debugging
- Code completion
- Documentation generation

6. Multilingual Understanding: The ability to understand and generate text in multiple languages.

Example

Input:

What is your name?

Spanish Output:

¿Cuál es tu nombre?

French Output:

Quel est votre nom ?

Benefits

- Global accessibility.
- Cross-language communication.
- Translation support.

Applications

- International chatbots
- Translation systems
- Language learning platforms

7. Knowledge Transfer: The ability to apply knowledge learned in one domain to another related domain.

Example

A model trained on scientific texts may apply logical reasoning skills to legal document analysis.

Advantages

- Better generalization.
- Reduced task-specific training.
- Improved adaptability.

8. Instruction Following: The ability to understand and execute natural language instructions accurately.

Example

Prompt:

Summarize this paragraph in three bullet points.

The model follows the exact instruction format.

Applications

- Virtual assistants
- Chatbots
- Content generation
- Educational tools

9. Creative Content Generation: The ability to generate original and coherent content.

Examples

- Stories
- Poems

- Essays
- Marketing content
- Scripts

Sample Prompt

Write a short story about a robot teacher.

The model can generate a complete story with characters and plot.

10. Tool Use and Agentic Behavior: Modern LLMs can use external tools and APIs to accomplish tasks.

Examples

- Web search
- Calculator usage
- Database querying
- Code execution

Benefits

- Extends model capabilities.
- Improves accuracy.
- Enables real-world task completion.

Advantages of Emergent Capabilities

1. Improved reasoning ability.
2. Better adaptability to new tasks.
3. Reduced need for task-specific training.
4. Enhanced user interaction.
5. Supports general-purpose AI applications.
6. Enables human-like language understanding.

Limitations and Challenges

- Models may generate incorrect information confidently.
- Models learn patterns rather than genuine comprehension.
- Training data may introduce social or cultural biases.
- Some emergent behaviors are difficult to anticipate.
- Large models require substantial resources.

Real-World Examples

GPT Series

- Few-shot learning
- Code generation

- Reasoning
- Creative writing

LLaMA Series

- Strong language understanding
- Multilingual capabilities
- Efficient reasoning

T5

- Task generalization
- Text-to-text problem solving

